BTCUSDT Market-Making Project

Backtesting an Avellaneda-Stoikov Strategy on Binance Spot Limit Order Book Data

Hadil

May 5, 2026

Abstract

This report presents the construction of a backtesting framework for a BTCUSDT market-making strategy using Binance Spot limit order book data. The strategy is based on the Avellaneda-Stoikov framework and is enriched with microstructure signals such as bid-ask spread, order book imbalance, top-of-book depth, and short-term volatility. The project includes data loading and cleaning, feature engineering, empirical calibration of fill probabilities as a function of quote distance from the mid-price, execution simulation, inventory management, terminal inventory liquidation, and a detailed PnL decomposition. The main objective is to understand under which conditions a market-making strategy can generate positive gross edge, and how this edge is affected by transaction costs, inventory exposure, and adverse selection.

1 Introduction

Market making consists of providing liquidity to the market by placing simultaneous limit orders on both the bid and ask sides. A market maker aims to buy at the bid and sell at the ask, thereby capturing part of the bid-ask spread. However, this activity is risky: the market maker can accumulate an unwanted directional position and can be executed precisely when the price is about to move against them. This phenomenon is known as adverse selection.

In this project, we build a backtester for a market-making strategy applied to BTCUSDT Spot. The core of the strategy is inspired by the Avellaneda-Stoikov model, which provides a theoretical quoting rule based on the mid-price, volatility, inventory, and risk-aversion parameters. Around this quoting engine, we add more realistic components: microstructure signals, risk filters, fill-probability calibration, transaction costs, and PnL decomposition.

The objective is not to claim that the resulting strategy is immediately tradable in production. The objective is to build a clear research framework that answers the following questions:

- Does the strategy generate positive spread capture?
- Does the PnL come from market making or from directional exposure to Bitcoin?
- How sensitive is the strategy to fees?
- Are the simulated fills favorable, or are they affected by adverse selection?
- Which components should be made more realistic to improve the backtest?

2 Data

The data used in this project consists of Binance Spot limit order book observations for the BTCUSDT symbol. The raw file contains the following columns:

Column	Description
timestamp	Observation date and time

symbol	Traded symbol, here BTCUSDT
best_bid_price	Best bid price
best_bid_qty	Quantity available at the best bid
best_ask_price	Best ask price
best_ask_qty	Quantity available at the best ask
mid_price	Mid-price
spread	Bid-ask spread

In our experiment, the dataset contains 20,000 rows. The first 30% of the observations are used for calibration, while the remaining 70% are used for out-of-sample backtesting. Therefore:

- 6,000 observations are used for calibration;
- 14,000 observations are used for backtesting.

The tick size used in the backtest is 0.01 USDT, which is consistent with the observed price grid in the BTCUSDT Spot data.

3 Microstructure Features

Several microstructure variables are computed from the order book data. These variables are used to determine whether the market is suitable for market making and to adjust quotes according to market conditions.

3.1 Mid-Price

The mid-price is defined as:

$$m_t = (b_t^{best} + a_t^{best}) / 2 \quad (1)$$

where b_t^best is the best bid and a_t^best is the best ask.

The mid-price is the central reference price. The strategy places quotes around this level, with adjustments based on inventory and microstructure signals.

3.2 Spread and Spread in Basis Points

The absolute spread is:

$$spread_t = a_t^{best} - b_t^{best} \quad (2)$$

To compare spreads across different price levels, we also compute the spread in basis points:

$$spread\_bps_t = 10{,}000 \times spread_t / m_t \quad (3)$$

The spread in basis points measures the relative cost of crossing the book. In our dataset, the spread is extremely tight: the median spread is around 0.0013 bps. This means that BTCUSDT is very liquid at the top of the book, but it also makes market making difficult when fees are significant.

3.3 Bid/Ask Imbalance

The top-of-book imbalance is defined as:

$$I_t = (Q_t^{bid} - Q_t^{ask}) / (Q_t^{bid} + Q_t^{ask}) \quad (4)$$

where Q_t^bid is the quantity at the best bid and Q_t^ask is the quantity at the best ask.

This quantity lies in [-1, 1]:

- I_t > 0 means that bid-side quantity is larger than ask-side quantity;
- I_t < 0 means that ask-side quantity is larger than bid-side quantity;
- I_t ≈ 0 means that the top of the book is balanced.

The intuition is the following: if the bid-side quantity is much larger than the ask-side quantity, the order book may reflect buying pressure, and the price may move upward in the short term. In that case, selling too aggressively at the ask is dangerous. Symmetrically, if the ask side dominates, buying too aggressively at the bid can expose the strategy to a short-term price decrease.

3.4 Order Book Depth

The top-of-book depth is defined as:

$$D_t = Q_t^{bid} + Q_t^{ask} \quad (5)$$

Low depth means that the book is thin and that the price may move quickly after an aggressive order. The strategy therefore avoids quoting when top-of-book depth is too low.

3.5 Returns and Short-Term Volatility

We compute log-returns of the mid-price:

$$r_t = \log(m_t) - \log(m_{t-1}) \quad (6)$$

Short-term volatility is estimated as the rolling standard deviation of these returns:

$$\sigma_t = Std(r_{t-w+1}, \ldots, r_t) \quad (7)$$

where w is the rolling window size, set to 100 observations in our experiments.

Volatility is a central input in the Avellaneda-Stoikov framework: when volatility increases, the risk of carrying inventory increases, and quotes should become more conservative.

4 The Avellaneda-Stoikov Model

The Avellaneda-Stoikov model provides a classical framework for optimal market making. The idea is to compute a reservation price and then place a bid and an ask around that price.

4.1 Reservation Price

The reservation price is given by:

$$r_t = m_t - q_t \gamma \sigma_t^2 T \quad (8)$$

where:

- m_t is the mid-price;
- q_t is the current inventory;

- γ is the risk-aversion coefficient;
- σ_t is the short-term volatility estimate;
- T is the effective horizon used in the formula.

The intuition is straightforward. If the strategy is long, meaning q_t > 0, the reservation price decreases. This makes the strategy more willing to sell and less willing to buy. If the strategy is short, meaning q_t < 0, the reservation price increases, which makes the strategy more willing to buy back inventory.

Thus, the model naturally introduces inventory control.

4.2 Optimal Spread

The theoretical total spread is approximated by:

$$\delta_t = \gamma \sigma_t^2 T + (2/\gamma) \log(1 + \gamma/\kappa) \quad (9)$$

where κ controls how fast the execution intensity decreases as quotes move farther from the mid-price.

In the Avellaneda–Stoikov model, kappa measures how quickly your chance of being filled decreases when you quote farther from the mid-price.

The first part of the spread,

$$\gamma \sigma_t^2 T \quad (10)$$

corresponds to inventory risk. The higher the volatility, the wider the optimal spread should be.

The second part,

$$(2/\gamma) \log(1 + \gamma/\kappa) \quad (11)$$

captures the trade-off between execution probability and profit per trade. Quoting close to the mid leads to more executions but smaller margins. Quoting farther away leads to fewer executions but higher profit per fill.

4.3 Bid and Ask Quotes

The raw quotes are computed as:

$$b_t = r_t - \delta_t/2, \quad a_t = r_t + \delta_t/2 \quad (12)$$

These quotes are then adjusted using microstructure signals and rounded to the tick size.

5 Inventory Skew and Imbalance Skew

5.1 Inventory Control

Inventory is one of the main risks faced by a market maker. If the strategy buys more often than it sells, it becomes long BTC. If the price then decreases, the strategy suffers a mark-to-market loss.

The Avellaneda-Stoikov model already partially addresses this issue through the reservation price. In the backtester, we also add constraints such as:

- if inventory exceeds an upper limit, the strategy stops placing bids;
- if inventory is too negative, the strategy stops placing asks;
- at the end of the backtest, any remaining inventory is liquidated.

Terminal liquidation is important: without it, a strategy may appear profitable simply because it ends the backtest with an unrealized position valued favorably at the mid-price

Terminal liquidation example

Suppose the strategy finishes the backtest with a positive BTC inventory, for example Q_T = 0.01 BTC. If PnL is computed as PnL_T = cash_T + Q_T m_T, the remaining inventory is valued at the mid-price m_T.

In practice, a long inventory cannot be closed at the mid-price. To liquidate a long BTC position, the strategy must sell at the best bid, with best bid < m_T < best ask. Therefore, marking the remaining position at the mid-price can slightly overestimate the realized PnL.

For example, if m_T = 100,000, best bid = 99,990, and Q_T = 0.01 BTC, then valuing inventory at the mid-price gives Q_T m_T = 0.01 x 100,000 = 1,000 USDT. Liquidating at the bid gives Q_T b_T = 0.01 x 99,990 = 999.90 USDT. The mid-price valuation is therefore 0.10 USDT too optimistic.

This is why the backtester forces terminal liquidation: if final inventory is positive, it is sold at the best bid; if final inventory is negative, it is bought back at the best ask. After this step, final inventory is zero and the final PnL is fully realized in USDT.

5.2 Imbalance Skew

Imbalance is used to adjust quotes according to short-term pressure observed in the book. We apply a skew of the form:

$$skew_t = \alpha_I I_t \cdot spread_t \quad (13)$$

where α_I controls the strength of the adjustment.

The quotes are then shifted as follows:

$$b_t \leftarrow b_t + skew_t, \quad a_t \leftarrow a_t + skew_t \quad (14)$$

If I_t > 0, the bid side is stronger than the ask side. This suggests potential upward pressure (sell quotes will become higher). Quotes are shifted upward: the bid becomes more aggressive and the ask becomes less aggressive, reducing the risk of selling too cheaply before an upward move.

If I_t < 0, quotes are shifted downward: the ask becomes more aggressive and the bid becomes less aggressive, reducing the risk of buying too high before a downward move.

The spread is used as a price scale: imbalance is dimensionless, whereas a quote skew must be expressed in price units.

Positive imbalance example

Consider the case Q_t^bid > Q_t^ask. There is more quantity waiting to buy than to sell at the top of the book, which can indicate short-term buying pressure. The price may move upward soon.

In this case, the imbalance skew shifts both quotes upward. For example, the bid may move from 99.90 to 99.95, while the ask may move from 100.10 to 100.15.

The higher bid is more aggressive: the strategy is willing to buy slightly higher because the market may rise. The higher ask is less aggressive: the strategy avoids selling too cheaply before a possible upward move.

Therefore, positive imbalance reduces the risk of selling BTC too cheaply before an upward price movement. Negative imbalance works in the opposite direction: quotes are shifted downward to avoid buying too high before a possible downward movement.

6 Fill Probability Calibration

A key part of the project is the empirical calibration of execution probabilities. Choosing fill probabilities arbitrarily would make the backtest unreliable.

6.1 Principle

At each time t, we construct virtual quotes at different distances from the mid-price. For a distance of d ticks:

$$b_t^{(d)} = m_t - d \cdot tick\_size, \quad a_t^{(d)} = m_t + d \cdot tick\_size \quad (15)$$

We then check whether the mid-price touches these levels over the next h observations.

For the bid, we compute:

$$\min_{1 \le u \le h} m_{t+u} \quad (16)$$

The virtual bid is considered touched if:

$$\min_{1 \le u \le h} m_{t+u} \le b_t^{(d)} \quad (17)$$

For the ask, we compute:

$$\max_{1 \le u \le h} m_{t+u} \quad (18)$$

and the virtual ask is considered touched if:

$$\max_{1 \le u \le h} m_{t+u} \ge a_t^{(d)} \quad (19)$$

This gives an approximation of the probability that an order placed d ticks away from the mid-price is executed within horizon h.

6.2 Exponential Fill Model

We then fit a model of the form:

$$p_{fill}(d) \approx p_0 \, e^{(-k_{fill} \, d)} \quad (20)$$

where:

- p_0 is the base probability of being executed near the mid;
- k_fill controls how fast the fill probability decreases with distance;
- d is the distance in ticks.

Taking logarithms gives:

$$\log p_{fill}(d) \approx \log p_0 - k_{fill} \, d \quad (21)$$

Equivalently, when the distance is expressed in price units, one can write log p_fill(d) approximately as log p0 - kappa x delta_price.

(where delta_price denotes the quote distance measured in price units)

Therefore, p_0 and k_fill can be estimated through a linear regression of log p_fill(d) on d.

In our experiment, the calibration gives approximately:

Parameter	Calibrated value
p_0	0.1745

k_fill	0.00181
κ	0.1813

The relation between k_fill and κ comes from the fact that k_fill is expressed per tick, while κ is expressed per price unit. Since:

$$\delta\_price = d \cdot tick\_size \quad (22)$$

we obtain:

$$\kappa = k\_fill / tick\_size \quad (23)$$

6.3 Limitations of This Calibration

This calibration is an approximation. It measures whether the price level was touched, but it does not guarantee that a real order would have been executed. In a real limit order book, execution also depends on:

- queue position;
- volume ahead of our order;
- market order flow;
- cancellations;
- latency.

Nevertheless, this approximation is useful as a first step because it makes the fill model data-driven.

7 Execution Model

In the first version of the backtester, fills are simulated probabilistically. For a quote at distance d ticks from the mid-price, we use:

$$p\_fill(d) = p_0\, e\text{^}(-k\_fill\, d) \quad (24)$$

At each timestamp, if the strategy places a bid or an ask, a uniform random variable is drawn. If this random variable is below the fill probability, the order is considered executed.

The sign conventions are:

- a fill at the bid is a buy, hence a positive signed quantity;
- a fill at the ask is a sell, hence a negative signed quantity.

If q_i is the signed quantity of trade i and p_i is its execution price, the impact on cash is:

$$\Delta cash\_i = -q\_i\, p\_i - fee\_i \quad (25)$$

This formula works for both buys and sells. If $q_i > 0$, the strategy buys and cash decreases. If $q_i < 0$, the strategy sells and cash increases.

8 Inventory Management and Terminal Liquidation

Inventory evolves according to:

$$Q_t = \Sigma_{i \leq t} q_i \quad (26)$$

The mark-to-market PnL is:

$$PnL_t = cash_t + Q_t m_t \quad (27)$$

At the end of the backtest, remaining inventory is liquidated:

- if Q_T > 0, the inventory is sold at the best bid;
- if Q_T < 0, the inventory is bought back at the best ask.

This liquidation avoids overestimating performance by valuing an unrealized final position at the mid-price. After liquidation, final inventory is zero and the PnL is fully realized.

9 PnL Decomposition

PnL decomposition is essential for understanding what the strategy is actually doing. We distinguish several components.

9.1 Total PnL

Total PnL is:

$$PnL_T = cash_T + Q_T m_T \quad (28)$$

After terminal liquidation, Q_T = 0, hence:

$$PnL_T = cash_T \quad (29)$$

9.2 Spread Capture

Spread capture measures the immediate gain obtained from buying below the mid-price or selling above the mid-price.

For a buy:

$$SC_i = q_i(m_i - p_i), \quad q_i > 0 \quad (30)$$

For a sell:

$$SC_i = |q_i|(p_i - m_i), \quad q_i < 0 \quad (31)$$

Positive spread capture indicates that the strategy buys below the mid and sells above the mid, which is the desired behavior for a market maker.

9.3 Inventory PnL

Inventory PnL measures the PnL generated by carrying a position while the mid-price moves:

$$InventoryPnL = \Sigma_t Q_{t-1}(m_t - m_{t-1}) \quad (32)$$

This component is important because it helps distinguish genuine market-making PnL from directional PnL. If a large part of the PnL comes from inventory, the strategy is exposed to market direction.

9.4 Fees

Fees are computed trade by trade:

$$fee_i = f |q_i| p_i \quad (33)$$

where **f is the fee rate**.

Total fees are:

$$Fees = \Sigma_i f |q_i| p_i \quad (34)$$

Fees are crucial in this project because the BTCUSDT spread is extremely tight. A strategy that appears profitable without fees may become unprofitable once realistic fees are included.

9.5 Adverse Selection

Adverse selection measures whether the price moves against the strategy after execution. For a horizon h, we compute:

$$AS_i = q_i(m_{i+h} - m_i) \quad (35)$$

For a buy, if the mid-price increases after the fill, this quantity is positive: the fill was favorable. If the mid-price decreases, it is negative: the strategy bought before a price drop.

For a sell, $q_i < 0$. If the mid-price decreases after the sale, the quantity is positive. If the mid-price increases, it is negative.

Thus, a negative average adverse-selection metric indicates that the strategy is being executed at unfavorable times.

10 Code Architecture

The project is organized modularly:

File	Role
data_loader.py	Data loading, validation, and cleaning
features.py	Microstructure feature engineering
calibration.py	Fill-probability and risk-threshold calibration
quote_engine.py	Avellaneda-Stoikov quotes and imbalance skew
risk.py	Risk filters and quote permissions
execution.py	Probabilistic fill simulation
strategy.py	Strategy logic, cash, inventory, and PnL
backtester.py	Backtest loop over all observations
pnl.py	PnL decomposition
metrics.py	Performance metrics
run_backtest.py	Main backtest script

This separation is important because it makes the project easier to read, test, and improve. For example, the execution model can be replaced without changing the quote engine, and the feature set can be improved without modifying the PnL logic.

Different steps of the backtest :

for each timestamp t:

 # 1. Observe market

- read best_bid, best_ask, bid_qty, ask_qty
- compute mid_price
- compute spread

 # 2. Compute features

- compute imbalance
- compute rolling volatility
- compute top-of-book depth

 # 3. Risk filters

decide whether quote_bid and quote_ask are allowed

if quote_bid or quote_ask:

 # 4. Avellaneda reservation price

reservation_price = mid_price - inventory * gamma * volatility**2 * horizon

 # 5. Avellaneda spread

- optimal_spread = gamma * volatility**2 * horizon \
  + (2 / gamma) * log(1 + gamma / kappa)
- bid = reservation_price - optimal_spread / 2
- ask = reservation_price + optimal_spread / 2

 # 6. Imbalance skew

- skew = alpha_imbalance * imbalance * market_spread
- bid = bid + skew
- ask = ask + skew

 # 7. Tick rounding

bid = round_down_to_tick(bid)

ask = round_up_to_tick(ask)

 # 8. Simulate fills

<u>if quote_bid:</u>

simulate bid fill

if filled:

buy base_order_size BTC

<u>if quote_ask:</u>

simulate ask fill

if filled:

sell base_order_size BTC

- Then the simulator draws a random number.
- <u>For the bid:</u>
- If random number < bid fill probability:

- bid is filled
- strategy buys 0.001 BTC
- For the ask:
- If random number < ask fill probability:
- ask is filled
- strategy sells 0.001 BTC

 # 9. Update cash and inventory

- update inventory
- update cash
- subtract fees

 # 10. Mark to market

- pnl = cash + inventory * mid_price

 # 11. Store results

- save state and trades

 # 12. Terminal liquidation

- if final inventory > 0:

sell remaining BTC at best bid

- if final inventory < 0:

buy back remaining BTC at best ask

- compute final PnL
- compute PnL decomposition

Calibration Module :

- The calibration module is used to make the strategy parameters data-driven instead of choosing them arbitrarily.
- It estimates two types of quantities from the calibration sample:

  1. Fill-probability parameters: p0, k_fill, and kappa

  2. Risk-filter thresholds: max_volatility, min_depth, and min_spread_bps

- The calibration is performed on the first 30% of the dataset, while the remaining 70% is used for the backtest. This avoids using future information when setting the strategy parameters.

1. Estimating Future Price Ranges

The first step is to compute, for each timestamp, the minimum and maximum mid-price observed over the next few observations.

For a given horizon, the function future_rolling_min computes the minimum future mid-price:

- future_min_mid(t) = min(mid_price(t+1), ..., mid_price(t+horizon))

Similarly, future_rolling_max computes the maximum future mid-price:

- future_max_mid(t) =max(mid_price(t+1), ..., mid_price(t+horizon))

These quantities are used to check whether a hypothetical bid or ask quote would have been touched in the near future.

The code uses shift(-1) to exclude the current observation and only look forward. The series is then reversed so that a rolling minimum or maximum can be computed over future values.

2. Virtual Bid and Ask Quotes

For each timestamp and each quote distance, the algorithm creates virtual bid and ask quotes around the mid-price.

For a distance of d ticks:

- virtual_bid = mid_price - d × tick_size
- virtual_ask = mid_price + d × tick_size

For example, if:

- mid_price = 76,300
- tick_size = 0.01
- distance = 5 ticks

then:

- virtual_bid = 76,300 - 5 × 0.01 = 76,299.95
- virtual_ask = 76,300 + 5 × 0.01 = 76,300.05

The idea is to ask: if we had placed a limit order at this distance from the mid-price, would it have been reached in the next few observations?

3. Fill Probability Estimation

For a virtual bid, the order is considered filled if the future mid-price goes below the virtual bid:

- future_min_mid <= virtual_bid

For a virtual ask, the order is considered filled if the future mid-price goes above the virtual ask:

- future_max_mid >= virtual_ask

This gives a Boolean value at each timestamp:

- True = the virtual quote would have been touched
- False = the virtual quote would not have been touched

The empirical fill probability is then computed as the average of these Boolean values.

For each distance and each horizon, the calibration computes:

- p_fill_bid = average bid fill probability
- p_fill_ask = average ask fill probability
- p_fill_mean = average of bid and ask fill probabilities

This produces a fill table containing the estimated probability of execution as a function of quote distance from the mid-price.

This is not a perfect execution model because it does not account for queue position, cancellations, or available volume ahead of our order.

However, it provides a simple and data-driven approximation of fill probabilities.

4. Exponential Fill Model

The strategy assumes that the probability of being filled decreases exponentially with the distance from the mid-price:

- $p\_fill(d) \approx p0 \times \exp(-k\_fill \times d)$

where:

- d is the quote distance in ticks
- p0 is the base fill probability
- k_fill controls how quickly fill probability decreases as the quote moves away from the mid-price

Taking logs gives:

- $\log(p\_fill(d)) \approx \log(p0) - k\_fill \times d$

This becomes a linear regression problem. The code fits a straight line between distance_ticks and log(p_fill_mean).

If:

- log(p_fill(d)) = intercept + slope × d

then:

- p0 = exp(intercept)
- k_fill = -slope

Before fitting the regression, probabilities equal to 0 or 1 are removed. This avoids numerical issues because log(0) is undefined, and probabilities equal to 1 are saturated values that can distort the fit.

In our experiment, the calibrated values were approximately:

- p0 = 0.1745
- k_fill = 0.00181

This means that a quote close to the mid-price has a relatively high probability of being filled, and the fill probability decreases gradually as the quote distance increases.

5. Conversion from k_fill to kappa

The parameter k_fill is estimated per tick, because the fill model uses distance measured in ticks.

However, the Avellaneda–Stoikov formula uses the liquidity parameter kappa, which is expressed per unit of price.

Since:

- distance_price = distance_ticks × tick_size

we have:

- exp(-k_fill × distance_ticks) = exp(-kappa × distance_price)

Therefore:

- kappa = k_fill / tick_size

With:

- k_fill = 0.00181
- tick_size = 0.01

we obtain approximately:

- kappa = 0.1813

This calibrated kappa is then used in the Avellaneda–Stoikov optimal spread formula.

6. Risk Threshold Calibration

The calibration module also estimates simple risk thresholds from the calibration data.

- First, **the volatility threshold** is computed as the 95th percentile of the rolling volatility:
    - max_volatility = quantile_95(rolling_volatility)

This means the strategy avoids quoting during the most volatile 5% of observations in the calibration sample.

In our experiment, this gave approximately:

    - max_volatility = 0.0000443

- Second, **the minimum depth threshold** is computed as the 10th percentile of the total top-of-book depth:
    - min_depth = quantile_10(total_depth)

This means the strategy avoids quoting when liquidity is in the lowest 10% of the calibration sample.

In our experiment, this gave approximately:

- min_depth = 2.617467

- Finally, **the minimum spread threshold** is computed from transaction costs:

  # I only quote if the observed market spread is bigger than fees by some margin.

  # I add a small buffer =0.5 to avoid adverse selection

    - latency
    - queue cost
    - slippage
    - rounding to tick
    - bad fills
- fee_bps_per_side = fee_rate × 10,000

  (fee_rate *100=fee_rate_*percent and we have 1 bps = 0.01% so fee_rate_percent/0.01=fee_rate *10^4)
- round_trip_fee_bps = 2 × fee_bps_per_side

  Why multiply by 2?

  Because a market-making cycle has two trades:

  1. buy BTC

  2. sell BTC

  So you pay fees twice : fee on the buy and fee on the sell
- min_spread_bps = round_trip_fee_bps + spread_buffer_bps

The buffer is added to account for risks not explicitly modeled, such as adverse selection, latency, queue cost, slippage, rounding to tick size, and bad fills.

In the fee-sensitivity experiments, this spread threshold was manually relaxed to 0.0 because BTCUSDT Spot has an extremely tight top-of-book spread, and the objective was to study how PnL changes under different fee assumptions.

7. Updating the Strategy Configuration

Once the calibration is complete, the strategy configuration is updated with the calibrated parameters:

- p0
- k_fill
- kappa
- max_volatility
- min_depth
- min_spread_bps

This is done using replace, which creates a new configuration object while preserving the other parameters.

The final output of the calibration module is a dictionary containing:

- fill_table: empirical fill probabilities by distance and horizon
- p0: calibrated base fill probability
- k_fill: calibrated fill decay per tick

- kappa: Avellaneda–Stoikov liquidity parameter
- max_volatility: calibrated volatility threshold
- min_depth: calibrated depth threshold
- min_spread_bps: calibrated spread threshold
- fit_ok: whether the exponential fit was successful

Summary

- The calibration module makes the backtest more realistic by estimating key parameters from the data.
- It first creates virtual bid and ask quotes at different distances from the mid-price, then checks whether these quotes would have been reached in the near future.
- From this, it estimates empirical fill probabilities and fits an exponential decay model.
- The resulting parameters p0, k_fill, and kappa are used in the execution model and Avellaneda–Stoikov quote engine.
- The module also calibrates volatility and depth thresholds using quantiles of the calibration sample.
- This allows the strategy to adapt its execution and risk filters to the observed BTCUSDT limit order book data.

Risk Filters and Strategy Parameters :

Before computing bid and ask quotes, the strategy applies a risk filter.

This module does not determine the quote prices directly.

Its role is to decide whether the strategy is allowed to quote on the bid side, the ask side, both sides, or neither side.

At each timestamp, the strategy checks five main elements:

- the bid-ask spread,
- short-term volatility,
- top-of-book depth,
- current inventory,
- order book imbalance.

First, the strategy checks whether **the spread is large enough:**

- spread_bps >= min_spread_bps

Second, the strategy checks whether **short-term volatility is acceptable**:

- rolling_volatility <= max_volatility
- The calibrated value used in the experiment was approximately:

  max_volatility = 0.0000443

- This prevents the strategy from quoting when short-term price movements are too unstable.

Third, the strategy checks whether the **top-of-book depth is sufficient**:

- total_depth >= min_depth
- The calibrated value was:

  min_depth = 2.617467

- This avoids quoting when the order book is too thin.

The strategy also **controls inventory**. The base order size is:

- base_order_size = 0.001 BTC
- and the inventory limit is:

  inventory_limit = 0.01 BTC

- If the strategy is already long more than 0.01 BTC, it stops placing bid orders and only allows ask quotes.
- If the strategy is short more than 0.01 BTC, it stops placing ask orders and only allows bid quotes.
- This prevents the market maker from accumulating excessive directional exposure.

Finally, the strategy uses order book imbalance as a simple **adverse-selection signal:**

imbalance = (best_bid_qty - best_ask_qty) / (best_bid_qty + best_ask_qty)

The threshold used is:

- imbalance_threshold = 0.4
- If the imbalance is greater than 0.4, the bid side is much stronger than the ask side. This may indicate upward price pressure, so selling at the ask becomes risky and the ask quote is blocked.
- If the imbalance is lower than -0.4, the ask side is stronger and the price may move downward, so buying at the bid becomes risky and the bid quote is blocked.

The output of the risk module is:

- quote_bid = True or False
- quote_ask = True or False
- reason = explanation

The main strategy parameters used in the backtest are

Parameter	Value	Meaning
symbol traded	BTCUSDT	Spot pair
tick_size BTCUSDT price increment	0.01	Minimum
lambda_as Stoikov risk-aversion parameter	0.1	Avellaneda–
horizon quoting horizon	1.0	Effective
base_order_size traded per fill	0.001 BTC	Quantity
inventory_limit absolute inventory	0.01 BTC	Maximum
imbalance_threshold used to block toxic sides	0.4	Threshold
alpha_imbalance imbalance-based quote skew	1.0	Strength of
fill_horizon for fill calibration	5	Horizon used
p0 base fill probability	0.1745	Calibrated
k_fill fill-probability decay	0.00181	Calibrated
kappa Stoikov liquidity parameter	0.1813	Avellaneda–

In summary, the risk module ensures that the strategy does not quote blindly. It avoids trading when the spread is too small, volatility is too high, depth is too low, inventory is already too large, or imbalance suggests strong short-term directional pressure.

11 Experimental Results

Traded instrument and accounting convention

The strategy trades the BTCUSDT spot pair. BTCUSDT represents the price of one BTC expressed in USDT. Therefore, the strategy buys and sells BTC, while all cash flows and PnL are measured in USDT.

A fill at the bid corresponds to buying BTC and paying USDT. A fill at the ask corresponds to selling BTC and receiving USDT. Consequently, inventory is measured in BTC, whereas cash and PnL are measured in USDT.

The risk-aversion parameter and imbalance-skew coefficient were initialized as baseline hyperparameters.

The fill-related parameters were calibrated empirically, while gamma and alpha_imbalance were kept fixed for the first backtest and can be optimized in future work through grid search or walk-forward validation.

lambda_as = 0.1 # Avellaneda risk aversion gamma

alpha_imbalance = 1.0 # quote skew strength

The strategy is backtested on 14,000 out-of-sample observations. The base order size is set to 0.001 BTC and the maximum inventory is set to 0.01 BTC.

The base order size is fixed at 0,001 BTC per quote, corresponding to approximately 76 USDT notional at the observed BTCUSDT price level.

So every time the bid is filled, the strategy buys about 76 USDT (At a BTC price around 76 000 USDT, this corresponds to a notional of: 0.001* 76 000 = 76 USDT worth of BTC. Every time the ask is filled, it sells about 76 USDT worth of BTC.

11.1 Zero-Fee Result

In the zero-fee case, the strategy produces the following results:

Metric	Value
Number of trades	955
Final PnL	14.68 USDT
Spread capture	4.19 USDT
Inventory PnL	10.49 USDT
Final inventory	0 BTC
Max drawdown	-1.73 USDT

The PnL is positive, but a significant part of it comes from inventory PnL. This means that the strategy benefited from favorable directional BTC exposure during the backtest.

11.2 Fee Sensitivity

We then perform a fee-sensitivity analysis. The results are:

Fee rate	Total fees	Final PnL	Non-annualized Sharpe
0.00000	0.0000	14.6801	0.0384
0.00001	0.7378	13.9423	0.0365
0.00005	3.6892	10.9909	0.0288
0.00010	7.3785	7.3016	0.0191
0.00020	14.7570	-0.0769	-0.0002

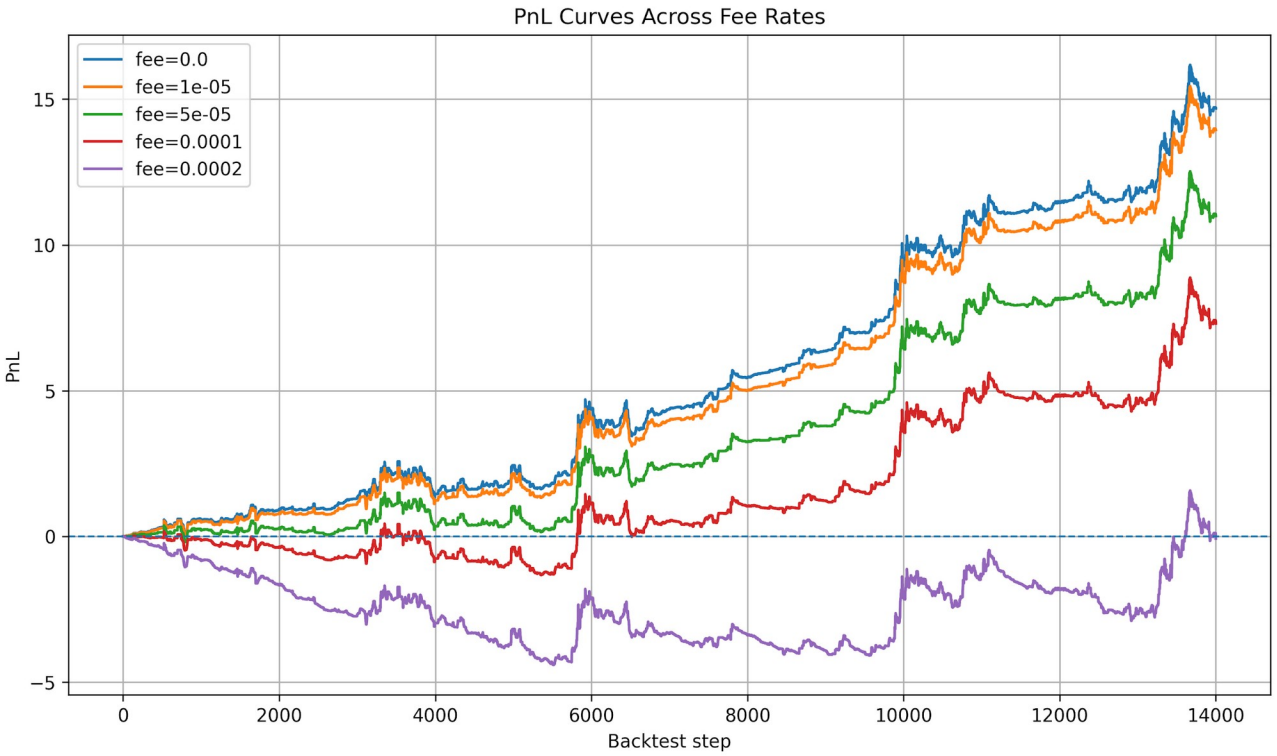

Figure: PnL curves across fee rates.

These results show that the strategy has a positive gross edge in the simplified backtest, but this edge is almost entirely consumed when the fee rate reaches 0.0002, i.e. approximately 2 bps per trade.

Break-Even Fee Rate Analysis

The break-even fee rate is the transaction cost level at which the strategy's net PnL becomes zero. In other words, it is the maximum fee rate the strategy can support before becoming unprofitable.

The net PnL can be written as:

- Net PnL = Gross PnL before fees - Total fees

The total fees are proportional to the total traded notional:

- Total fees = fee_rate × total_traded_notional

where the total traded notional is the total USDT value of all executed trades:

- total_traded_notional = sum(quantity × execution_price)

In our backtest, each trade has a size of approximately 0.001 BTC. With BTC trading around 76,000 USDT, each trade corresponds to roughly:

$0.001 \times 76{,}000 \approx 76$ USDT

Across the full backtest, the total traded volume was approximately 0.964 BTC, corresponding to a

- total traded notional of about: 73,785 USDT

The gross PnL before fees was:

- Gross PnL before fees = 14.6801 USDT

Therefore, the break-even fee rate is:

break_even_fee_rate = gross_pnl_before_fees / total_traded_notional (: Σ_i |q_i| p_i)

Using the observed values:

- break_even_fee_rate = 14.6801 / 73,784.85 ≈ 0.000199

This corresponds to approximately:

- 0.0199% per trade ≈ 1.99 basis points per trade

Equivalently, using the run with fee_rate = 0.0002, we observed:

- Total fees = 14.7570 USDT
- Final PnL = -0.0769 USDT

This confirms that the strategy is very close to break-even when the fee rate is around 0.0002.

The interpretation is that, in this simplified backtest, the strategy generates about 2 basis points of gross edge per traded notional. If transaction costs are below this level, the strategy remains profitable. If transaction costs are above this level, the strategy becomes unprofitable.

This result highlights the importance of transaction costs in high-frequency market-making strategies. Since the BTCUSDT top-of-book spread is extremely tight, even small fees can consume most of the strategy's gross edge.

12 Economic Interpretation

The results highlight several important points.

- First, the strategy does capture spread: spread capture is positive. This means that the quotes are placed coherently around the mid-price.
- Second, a large part of the PnL comes from inventory PnL. This means that the strategy is not a perfectly neutral market maker: it sometimes carries directional exposure. This is not necessarily bad, but it must be clearly identified. A strategy that profits mainly from inventory exposure is closer to a directional strategy than to pure market making.
- Third, profitability is highly sensitive to fees. On BTCUSDT Spot, the top-of-book spread is extremely tight. Therefore, even a small error in the fill model or in the fee assumption can completely change the conclusion.
- Fourth, adverse selection is a central metric. If fills tend to be followed by unfavorable mid-price moves, the strategy is providing liquidity at the wrong times.

13 Backtest Limitations

The current backtest is a first version and is intentionally simplified. The main limitations are the following.

13.1 Simplified Fill Model

The execution model is probabilistic and depends only on quote distance from the mid-price. It does not account for:

- queue position;
- actual volume consumed at the bid and ask;
- cancellations;

- market order flow;
- latency.

A more realistic model should simulate the position of the order in the queue and determine whether aggressive flow is sufficient to reach it.

13.2 Liquidity Assumption
The backtest assumes that the order is executed at the simulated quote with the desired quantity. In reality, partial execution is possible.

13.3 Fees and Rebates
Fees are modeled as a simple fixed percentage. In practice, fees depend on VIP level, maker/taker status, and may include rebates. For a real market maker, the fee structure is crucial.

For Binance Spot BTCUSDT, there are two different notions:

- **Standard VIP spot fees**: this is the normal maker/taker fee schedule.
- **Liquidity Provider / maker rebate programs:** these are special programs for eligible market makers and may give actual maker rebates.

13.4 Limited Test Period
The dataset is relatively short. A more robust validation should use multiple days or weeks and include different market regimes: calm, volatile, directional, and mean-reverting.

14 Possible Improvements
Several natural improvements can be made to the project.

14.1 Queue-Based Execution Model
The most important improvement would be to replace the probabilistic execution model with a queue-based model. The idea would be to track the volume ahead of our order at a given price level and simulate execution only if aggressive orders consume that volume.

14.2 Adverse-Selection Filter
A toxicity score could be added using:

- imbalance;
- short-term volatility;
- rapid mid-price changes;
- order book depth;
- large spread movements.

The strategy could stop quoting or widen quotes when the toxicity score is too high.

14.3 Parameter Optimization
Parameters such as γ, inventory limits, imbalance-skew intensity, and order size could be optimized on a calibration period and then validated out of sample.

14.4 Stricter Inventory Control
Since a large part of the PnL comes from inventory PnL, it would be useful to test stricter inventory controls:

- reducing the inventory limit;
- applying stronger skew when inventory approaches the limit;
- partially liquidating inventory when it remains on the same side for too long.

14.5 Multi-Period Backtesting

The strategy should be tested on several disjoint periods to verify robustness and avoid overfitting to a single market regime.

15 Conclusion

This project developed a complete backtesting framework for an Avellaneda-Stoikov market-making strategy applied to BTCUSDT Spot. The framework includes order book data loading, microstructure feature computation, empirical fill-probability calibration, inventory-aware quote computation, execution simulation, cash and inventory management, terminal liquidation, and PnL decomposition.

The results show that the strategy generates positive gross edge in a simplified environment. However, this edge is highly sensitive to transaction costs. With a fee rate close to 0.0002, the strategy becomes close to break-even. In addition, a significant part of the PnL comes from inventory exposure, showing that the strategy is not yet a pure neutral market maker.

The main conclusion is that market-making profitability does not depend only on the quoting formula. It depends crucially on execution quality, fees, adverse selection, inventory management, and the ability to avoid toxic market regimes. The developed framework therefore provides a solid foundation for moving toward a more realistic backtest incorporating queue position, market order flow, and more advanced toxicity filters.